

Java Classes and Interfaces

Implementation and explanation of the 8 given class/interface inheritance cases

Basic Concepts

Class: A class is a blueprint for creating objects. It can contain variables, constructors and methods. A class can extend only one class because Java does not support multiple inheritance of classes.

Interface: An interface defines a contract that implementing classes agree to follow. A class can implement multiple interfaces. An interface can extend one or more interfaces.

extends: Used for class-to-class inheritance and interface-to-interface inheritance. **implements:** Used when a class provides implementations for an interface.

No.	Given statement	Valid in Java?	Meaning
1	C1 extends C2	Yes	Class C1 inherits from class C2.
2	C1 extends C2, C3	No	A class cannot extend multiple classes.
3	C1 implements I1	Yes	Class C1 implements interface I1.
4	C1 implements I1, I2	Yes	Class C1 implements multiple interfaces.
5	C1 implements C2 extends I1	No	Invalid syntax; implements requires interfaces, and extends cannot follow it.
6	I1 extends I2	Yes	Interface I1 inherits from interface I2.
7	I1 implements C1	No	An interface cannot implement a class.
8	I1 extends I2, I3	Yes	Interface I1 can extend multiple interfaces.

1. C1 extends C2

This is valid single inheritance. C1 is the child/subclass and C2 is the parent/superclass.

Program:

```
class C2 {
    void showC2() {
        System.out.println("Method of C2");
    }
}

class C1 extends C2 {
    void showC1() {
        System.out.println("Method of C1");
    }
}

public class Case1 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.showC2();
        obj.showC1();
    }
}
```

Output:

Method of C2
Method of C1

2. C1 extends C2, C3

This is not valid Java. Java does not allow a class to extend two classes. The correct approach is to use one superclass and, if needed, implement multiple interfaces.

Program:

```
// Invalid:
// class C1 extends C2, C3 { }

// Correct alternative:
class C2 {
    void showC2() {
        System.out.println("Method of C2");
    }
}

class C3 {
    void showC3() {
        System.out.println("Method of C3");
    }
}
```

// Java cannot make C1 extend both C2 and C3.
// Use composition or interfaces instead.

Result:

Compilation error for: class C1 extends C2, C3

3. C1 implements I1

This is valid. A class implements an interface and must provide implementations for its abstract methods.

Program:

```
interface I1 {
    void display();
}

class C1 implements I1 {
```

```

    public void display() {
        System.out.println("I1 method implemented by C1");
    }
}

public class Case3 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.display();
    }
}

```

Output:

I1 method implemented by C1

4. C1 implements I1, I2

This is valid multiple-interface implementation. C1 must implement the required methods of both interfaces.

Program:

```

interface I1 {
    void method1();
}

interface I2 {
    void method2();
}

class C1 implements I1, I2 {
    public void method1() {
        System.out.println("I1 method");
    }

    public void method2() {
        System.out.println("I2 method");
    }
}

public class Case4 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method1();
        obj.method2();
    }
}

```

Output:

I1 method

I2 method

5. C1 implements C2 extends I1

This exact declaration is invalid. C2 is written as though it is an interface, but the name alone does not make it one. Also, Java class declarations use 'extends' before 'implements'. A valid pattern is: class C1 extends C2 implements I1, where C2 is a class and I1 is an interface.

Program:

```

interface I1 {
    void show();
}

class C2 {
    void parentMethod() {
        System.out.println("Method of C2");
    }
}
// Valid syntax:

```

```

class C1 extends C2 implements I1 {
    public void show() {
        System.out.println("Method of I1");
    }
}

public class Case5 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.parentMethod();
        obj.show();
    }
}

Output:
Method of C2
Method of I1

```

6. I1 extends I2

This is valid interface inheritance. I1 becomes a subinterface of I2 and inherits its contract.

Program:

```

interface I2 {
    void method2();
}

interface I1 extends I2 {
    void method1();
}

class C1 implements I1 {
    public void method1() {
        System.out.println("I1 method");
    }

    public void method2() {
        System.out.println("I2 method");
    }
}

public class Case6 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method1();
        obj.method2();
    }
}

Output:
I1 method
I2 method

```

7. I1 implements C1

This is invalid Java. An interface cannot implement a class. If I1 should inherit from another interface, use 'extends'. If C1 is a class, then C1 can implement I1.

Program:

```

// Invalid:
// interface I1 implements C1 { }

// If C1 is a class and I1 is an interface:
interface I1 {
    void display();
}

class C1 implements I1 {
    public void display() {

```

```

        System.out.println("C1 implements I1");
    }
}

public class Case7 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.display();
    }
}

```

Result:

The statement 'interface I1 implements C1' gives a compilation error.

8. I1 extends I2, I3

This is valid multiple interface inheritance. An interface can extend more than one interface. A class implementing I1 must satisfy the methods inherited from I2 and I3.

Program:

```

interface I2 {
    void method2();
}

interface I3 {
    void method3();
}

interface I1 extends I2, I3 {
    void method1();
}

class C1 implements I1 {
    public void method1() {
        System.out.println("I1 method");
    }

    public void method2() {
        System.out.println("I2 method");
    }

    public void method3() {
        System.out.println("I3 method");
    }
}

public class Case8 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method1();
        obj.method2();
        obj.method3();
    }
}

```

Output:

```

I1 method
I2 method
I3 method

```

Important Rules to Remember

- Class extends class: A class can extend only one class.
- Class implements interface: A class can implement one or many interfaces.
- Interface extends interface: An interface can extend one or many interfaces.
- Interface implements class: Not allowed.
- Class implements class: Not allowed.
- Class extends interface: Not allowed; a class implements an interface.
- Correct combined syntax: `class C1 extends C2 implements I1, I2 { }`
- Multiple interface inheritance: `interface I1 extends I2, I3 { }`

Quick Summary

Case	Correct/Incorrect	Main concept
1	Correct	Single class inheritance
2	Incorrect	Multiple class inheritance is not supported
3	Correct	Single interface implementation
4	Correct	Multiple interface implementation
5	Incorrect as written	Use: extends class, then implements interface(s)
6	Correct	Interface inheritance
7	Incorrect	Interface cannot implement a class
8	Correct	Multiple interface inheritance

Note: Each valid case is demonstrated with a small Java program. Invalid cases are shown with the reason they fail and, where useful, a corrected Java pattern.